

DART - Digital Asset Routing and Transformation

DART (Digital Asset Routing and Transformation) is a desktop application built with Python and [Flet](#) for managing digital asset workflows in Grinnell College Libraries. Using DART the user selects a 'truth' set of digital asset files, then applies up to 6 powerful functions to manage the workflow.

Key functions, in typical order of execution, include:

1. analyzing assets and assigning unique IDs to each while optionally grouping like-named assets into compound objects,
2. creating initial control/management metadata around the new IDs,
3. generating small and thumbnail derivatives,
4. deposit objects and derivatives properly into managed, web-addressable Azure Blob Storage,
5. optionally engage Seeklight to generate additional or alternative metadata, and
6. manage the process of selectively merging new metadata with old to complete our routing and transformation workflow.

Working with AI

This repository now includes project-specific AI guidance files:

- [AGENTS.md](#) — repository rules and architectural guidance for coding agents
- [HUMANS.md](#) — how contributors can prompt and review AI effectively on DART
- [CLAUDE.md](#) — pointer file so Claude-based tools inherit the same instructions
- [copilot-instructions.md](#) — DART-specific working guidance for GitHub Copilot sessions

If you use AI to work on DART, start with these files before making changes. Keep [AGENTS.md](#) as the source of truth when repository rules evolve.

Core Mission

DART is focused on providing a valid import/ingest-compatible CSV metadata file using the digital objects and their original filenames as the "source of truth". Each object is given a unique Digital.Grinnell identifier and DART can help maintain those while directing files to proper long-term/preservation storage.

CSV Metadata Workflow:

- Define a **core metadata CSV** file that serves dual purposes:
 - **Column structure template:** Defines required/recommended CollectionBuilder fields
 - **Master metadata file:** Acts as the "source of truth" for your collection
 - Can be located anywhere (inside or outside working directory)
- Generate new metadata from asset batches using the core CSV's structure
- **Isolated Working Files:** DART automatically creates [.DART-working-directory](#) subfolder
 - Keeps DART_export CSV files separate from project files
 - Hidden folder doesn't clutter your main directory
 - Functions 2, 3, and 4 automatically use this subfolder
- **Smart Merging:** Function 4 always updates the core CSV (wherever it's located)

- No confusion about which file is the master
- Working files stay in isolated subfolder
- Maintain consistency and avoid duplicate identifiers across workflow phases

Identifier format note: Epoch cutoff **1782237851** marks the introduction of the optional **dg_prefix** setting. IDs with earlier epoch values are legacy **dg_<epoch>** IDs. IDs generated after that cutoff may appear either as **dg_<epoch>** when the prefix is blank or as **<prefix>_dg_<epoch>** when a prefix is configured.

Recommended collection identifier convention: When using **dg_prefix**, choose the stable CollectionBuilder collection identifier, such as **tdps**, **student-life**, or **oral_history**. It may be any length and can use letters, numbers, hyphens, and underscores. For the unified Digital.Grinnell CollectionBuilder site, use the same identifier for the collection slug and Azure asset-path directory, such as **objs/student-life**, so IDs and published assets remain aligned.

OHM-data Mode

Set **process_OHM_data** to **true** in Function 0 for packaged Oral History Manager data. Select the project root containing **OHM-data**, or select the **OHM-data** directory itself.

- Function 1 recursively finds only **.mp3** files.
- Existing **dg_...** values in MP3 filenames are preserved; **dg_prefix** is prepended when configured.
- Function 2 creates one standalone **type=transcript** record per MP3. The record uses **display_template=transcript**, has a blank **parentid**, stores the normalized transcript filename in **object_transcript**, and uses the MP3 Azure URL in **object_location**.
- The transcript CSV is copied to the target project's **_data/transcripts** directory and uploaded to the parallel Azure **transcripts** path.
- Other sibling files are excluded from the metadata CSV but uploaded to Azure's **objs** path with normal Hot-tier storage. A **dg_** sibling uses **<prefix>_<dg_basename>.<extension>**; other siblings use **<prefix>_<basename>_<dg_basename>.<extension>**.
- Function 3 does not generate OHM derivatives because sibling files are not metadata records.

Purpose

DART provides a comprehensive platform for digital asset management workflows:

- **Select source assets** from a folder (truth set) or CSV manifest
- **Process and transform** digital assets through multiple workflow phases
- **Generate derivatives** in various formats and sizes
- **Modify and update** CSV metadata files
- **Route assets** to output directories with customizable organization
- **Track progress** with detailed logging and status reporting

Features

Core Platform Features

- **Persistent Settings:** Automatic saving/loading of window position, directories, and user preferences
- **Persistent File Selection:** Selected files are remembered across app restarts - no need to re-select
- **Permanent ID Assignment:** Files receive unique permanent identifiers that never change once assigned: legacy `dg_<epoch>` by default, or optional `<prefix>_dg_<epoch>` when `dg_prefix` is configured in Function 0
- **CSV Metadata Management:** Template-based CSV generation with intelligent merging into master metadata file
- **Isolated Working Directory:** Automatic `.DART-working-directory` subfolder for temporary files
 - DART_export CSV files written to hidden subfolder
 - Keeps working files separate from project files
 - Core metadata CSV can be located anywhere
 - Merges always update the core CSV (not working directory)
- **CollectionBuilder Compatibility:** Validates CSV structure for required fields (objectid, original_file_name)
- **Azure Blob Storage Integration:** Automatic file uploads with encrypted connection strings
- **Kill Switch:** Emergency stop button for batch operations (stops cleanly without data corruption)
- **Professional Logging:** Timestamped log files in `{working_folder}/logfiles/` with real-time display
 - All UI messages automatically written to persistent log files
 - Intelligent log level detection (ERROR, WARNING, INFO, DEBUG)
 - Clickable log viewer in results dialogs
 - Complete audit trail of all operations
- **Function Management:** Icon-enhanced dropdown with usage tracking and workflow ordering
 - **Workflow Tracking Indicators:** Visual progress markers in function dropdown
 - "✓ Last" shows most recently executed function
 - "► NEXT ◀" highlights suggested next step in workflow sequence
 - Auto-updates after each function execution
 - Workflow sequence: 1 (Analyze) → 2 (Export) → 3 (Derivatives) → 4 (Compare/Merge)
- **Help Mode:** Built-in markdown help viewer for each function with copy-to-clipboard
- **Smart Folder Management:** Collapsible folders section to maximize screen space
- **File Selection:** Dedicated file picker with persistence (separate from directories)
- **Status & Log Output:** Professional status display with copy/paste and log management
- **Secure Settings:** Encrypted storage for API keys and credentials

Workflow Functions

- **Function 0**  : App Settings with encrypted sensitive fields
 - **Automated Workflow:** Enable `automatic_four` to automatically execute Functions 2, 3, and 4 after Function 1 completes
 - Creates a seamless workflow from asset analysis through metadata merge
 - Stops immediately if any errors occur
 - Automatically resets to `false` at start of each new session (opt-in per session)
 - **Optional Collection Identifier:** Set `dg_prefix` to generate new IDs as `<collection>_dg_<epoch>` for multi-collection uniqueness. Collection identifiers may be

any length and can contain letters, numbers, hyphens, and underscores

- **Function 1**  : Analyze digital assets and generate standard DG identifiers (dg_)
 - Creates compound objects for related file groups (optional)
 - Permanent ID assignment with folder-based compound tracking
 - When **automatic_four** is enabled, automatically triggers Functions 2, 3, and 4
- **Function 2**  : Export Assets to CSV and Azure
 - Generates CollectionBuilder-compatible metadata files
 - Auto-populates objectid, original_file_name, parentid, display_template, format, and object_location fields
 - Supports compound object export with parent/child relationships
 - Maps file types to CollectionBuilder layouts (image/video/audio/pdf/compound_object)
 - **Azure Blob Storage integration:** Automatically uploads files and generates object_location URLs
 - **Auto-creates Azure containers:** No manual Azure Portal setup required
 - Files uploaded with object identifiers as filenames (e.g., **dg_1715614222.jpg** or **tdps_dg_1715614222.jpg**)
 - **Optional overwrite mode:** Set **overwrite_existing_azure_files=true** in Function 0 to replace existing Azure blobs instead of skipping them
 - **Kill Switch:** Emergency stop for long-running Azure uploads (stops cleanly after current file)
 - Timestamped exports to **.DART-working-directory** subfolder
- **Function 3**  : Generate Derivatives for CSV and Azure
 - Creates small (800x800) and thumbnail (400x400) image derivatives
 - Uploads derivatives to Azure Blob Storage (/smalls/ and /thumbs/ folders)
 - **Auto-creates derivative containers:** Automatic /smalls/ and /thumbs/ container setup
 - **Smart skip existing:** Checks Azure and skips files with existing derivatives by default (fast re-runs)
 - **ICC-aware TIFF handling:** Embedded grayscale/scanner profiles are converted to sRGB before JPEG derivatives are written
 - **Optional overwrite mode:** Set **overwrite_existing_azure_files=true** in Function 0 to regenerate and replace existing derivatives in Azure
 - Automatically populates image_small and image_thumb CSV columns
 - Maintains aspect ratios, handles EXIF orientation and transparency
 - Reads from and writes to **.DART-working-directory** subfolder
 - **Clickable log viewer:** Results dialog includes link to open detailed log in popup
 - **Kill Switch:** Emergency stop for long-running derivative generation
- **Function 4**  : Compare and Merge CSV Files
 - **CSV comparison using csvdiff tool:** Fast, accurate comparison with JSON output
 - Compare two CSV files by original_file_name with detailed change tracking
 - Auto-selects newest DART_export CSV from **.DART-working-directory** subfolder
 - Classifies records: added (new), removed (missing in new), changed (different values)
 - Generates JSON diff results and text summary
 - **Interactive merge with preview:** Field-level checkboxes (checked by default), data loss fields disabled/grayed requiring manual enable
 - **Smart merge behavior:** Always updates core CSV file (wherever it's located)
 - Case-sensitive comparison with whitespace normalization
 - **Preserves core CSV structure:** Never reorders rows from core metadata CSV

- **Function 5** 🔍 : Engage Seeklight Metadata Generation
 - Transform Seeklight-generated metadata to DART core metadata format
 - File picker with remembered directory for Seeklight CSV exports
 - Smart matching with existing object IDs based on filenames
 - JSON-based field mapping template (`seeklight_mapping_template.json`)
 - Automatic generation of new IDs for unmatched files
 - Customizable field mappings and default values
 - Timestamped output in `.DART-working-directory` subfolder
 - Complete Seeklight web workflow documentation in help system
- **Function 6** 🔗 : Compare and Merge Seeklight CSV
 - Compare Seeklight-transformed CSV files with core metadata
 - Basename-to-objectid matching (Seeklight `original_file_name` basename matched to core objectid)
 - Interactive merge with field-level checkboxes for granular control
 - Data loss protection: unchecks fields where Seeklight value is empty
 - Selective merging: choose exactly which changes to accept
 - Automatic backup creation before merge
 - Shows matched, new, and changed records with clear indicators
- **Function 9** 🖨️ : Display system information

Additional asset processing functions are added as needed for your specific workflow.

Quick Start

Installing from DMG (macOS)

DART for macOS features an **automated installer** that requires no admin permissions.

What you receive: `DART_v2.0.dmg`

Installation steps:

1. **Open the DMG** (double-click it)
2. **Run the installer:**
 - Double-click: `Install DART.command`
 - **First time only:** If blocked, right-click → Open → click "Open"
3. **Follow prompts:**
 - Installation completes in seconds
 - DART installs to `~/DART/` (your home directory)
 - A launcher appears on your Desktop
4. **Eject the DMG**
5. **Launch DART:**
 - Double-click `Launch DART` on your Desktop
 - Or navigate to `~/DART/` and double-click `Launch DART.command`

First launch:

- Terminal window opens and installs dependencies (1-2 minutes, first time only)
- DART application window opens when ready
- Keep Terminal open while using DART

Why home directory?

- No admin permissions required
- Works on all systems, including managed/restricted work computers
- Clean, predictable location: `~/DART/`

Prerequisites: Python 3.8+ must be installed

- Download from python.org/downloads
- Or via Homebrew: `brew install python`
- Verify: `python3 --version`

 **For detailed instructions, see [INSTALLATION.md](#)**

Installing from ZIP (Development)

1. Clone or download this repository

```
git clone https://github.com/Digital-Grinnell/DART.git
cd DART
```

2. Install common-DG-utilities (required dependency)

```
cd ..
git clone https://github.com/Digital-Grinnell/common-DG-utilities.git
cd DART
```

3. Run the launcher:

- Double-click: `scripts/run.bat`

4. First launch:

- Command Prompt opens and installs dependencies (1-2 minutes, first time only)
- DART application window opens when ready
- Keep Command Prompt open while using DART

Prerequisites: Python 3.8+ must be installed with "Add Python to PATH" checked

- Download from python.org/downloads
- Verify: `python --version` in Command Prompt

Optional: Create desktop shortcut: Right-click `scripts/run.bat` → Send to → Desktop

📖 For detailed instructions, see [INSTALLATION.md](#)

1. Clone or copy this repository

```
cd /path/to/your/projects
cp -r DART my-new-app
cd my-new-app
```

2. Run the application

```
# macOS/Linux
```

`./run.sh`

Windows

`scripts\run.bat`

The run scripts automatically:

- Create a Python virtual environment
- Install dependencies

On macOS/Linux, `./run.sh` is the compatibility launcher in the project root and forwards to `scripts/run.sh`.

- Launch the application

Requirements

- **Python 3.8+**
- **Flet 0.25.2** (installed automatically by run scripts)
- **cryptography** (for encrypted settings)
- **azure-storage-blob 12.19.0** (for Azure Blob Storage uploads)
- **common-DG-utilities** (included in DMG/ZIP packages; required for source builds)
- Contains shared Digital.Grinnell utility functions
- Auto-included by build scripts from sibling directory

All dependencies are installed automatically by the run scripts.

Project Structure

DART/ |—— run.sh # Root compatibility launcher for macOS/Linux |—— app.py # Main application
|—— scripts/ # Stand-alone launchers and helper scripts | |—— run.sh # macOS/Linux launcher |

|—— run.bat # Windows launcher | |—— build_dmg.sh # macOS installer builder | |——
 build_windows_zip.sh # Windows package builder | |—— rename_metadata_field.py | |——
 fix_config_csv_fields.sh | |—— batch_rename_dublin_core.sh | |—— cleanup_old_backups.sh | |——
 diagnose_rename_changes.sh | |—— migrate_legacy_working_files.py |—— VERSION # Version
 number file |—— python_requirements.txt # Python dependencies |—— .gitignore # Git exclusions
 |—— LICENSE # MIT License |—— CHANGELOG.md # Version history |—— README.md # This file
 (full documentation) |—— INSTALLATION.md # Detailed installation guide |—— QUICKSTART.md #
 Quick reference guide |—— FUNCTION_0_APP_SETTINGS.md # Help docs for Function 0 |——
 FUNCTION_1_ANALYZE_ASSETS.md # Help docs for Function 1 |—— FUNCTION_2_EXPORT_CSV.md #
 Help docs for Function 2 (CSV and Azure export) |—— FUNCTION_3_GENERATE_DERIVATIVES.md #
 Help docs for Function 3 (Derivatives) |—— FUNCTION_4_COMPARE_MERGE_CSV.md # Help docs for
 Function 4 (Compare/Merge) |—— RENAME_METADATA_FIELD.md # Documentation for field renaming
 tools |—— FIXING_RENAME_ISSUES.md # Troubleshooting guide for renaming problems |——
 FIX_COLON_TO_UNDERSCORE.md # Guide for normalizing legacy dc: / dc. / dc_ field variants |——
 common_dg_utilities/ # Shared utility functions (auto-included in builds)

Runtime Files

When you run the application, these are created automatically:

~/DART-data/ |—— persistent.json # Saved settings and state |—— encryption_key # Encryption key
 for sensitive settings

{working_folder}/ |—— .DART-working-directory/ # Isolated DART-generated artifacts | |——
 dart_settings.json # Per-folder app settings | |—— DART_export_YYYYMMDD_HHMMSS.csv # CSV
 exports from Function 2 | |—— DART_export_with_derivatives_.csv # *Function 3 outputs* | |——
 DART_seeklight_transformed_.csv # Function 5 transformed output | |—— csvdifff_result_.json #
Function 4 diff output | |—— csvdifff_summary_.txt # Function 4 summary output | |——
 *.backup_YYYYMMDD_HHMMSS # Merge backups (Functions 4 and 6) | |—— temp_derivatives/ #
 Temporary derivative staging |—— logfiles/ |—— dart_YYYYMMDD_HHMMSS.log # Application logs

****Note:**** Log files are created in your working/outputs folder (set in the
 UI) to keep logs organized with the project data they relate to. On initial
 startup before a working folder is set, logs temporarily go to `~/DART-
 data/logfiles/`.

****Working Directory Structure:**** DART automatically creates a `.DART-
 working-directory` hidden subfolder to keep temporary DART_export files
 separate from your project files (e.g., CollectionBuilder files). Your core
 metadata CSV remains wherever you've placed it (often in `\_data/` for
 CollectionBuilder projects).

Customizing DART for Your Application

1. Rename the Application

Update these items throughout the codebase:

- `page.title` in `app.py`
- Data directory name (`DART-data` → `YourApp-data`)
- Window title and header text
- Script headers in `scripts/run.sh` and `scripts/run.bat`
- README title and descriptions

2. Add Your Own Functions

To add a new function, follow the OHM-proven pattern:

****a) Create the function handler in `app.py`:****

```
```python
def on_function_4_your_feature(e):
 """Function 4: Your custom feature description."""
 storage.record_function_usage("Function 4")

 # Access current directory if needed
 if not current_directory or not current_directory.exists():
 update_status("Error: Please select an input folder first",
is_error=True)
 return

 # Your implementation here
 # ... do work ...

 update_status("Your feature completed successfully")
 add_log_message("Function 4 completed")
 logger.info("Function 4: Completed")
```

### **b) Add to the active\_functions list:**

```
active_functions = [
 "function_1_list_files",
 "function_2_count_files",
 "function_3_system_info",
 "function_4_your_feature", # Add this
]
```

### **c) Register in the functions dictionary:**

```
functions = {
 # ... existing functions ...
 "function_4_your_feature": {
 "label": "4: Your Custom Feature",
 "icon": "🔴", # Pick an emoji icon
 "handler": on_function_4_your_feature,
 "help_file": "FUNCTION_4_YOUR_FEATURE.md"
```

```
 },
}
```

#### d) Create help documentation:

Create `FUNCTION_4_YOUR_FEATURE.md` with markdown documentation. The template automatically:

- Shows help in a dialog when Help Mode is enabled
- Provides copy-to-clipboard functionality
- Displays the function's icon and label

### 3. Add Dependencies

If your functions need additional Python packages:

1. Add them to `python_requirements.txt`:

```
flet==0.25.2
flet-desktop==0.25.2
your-package>=1.0.0
```

2. Import them in `app.py`:

```
try:
 import your_package
 YOUR_PACKAGE_AVAILABLE = True
except ImportError:
 YOUR_PACKAGE_AVAILABLE = False
```

3. Check availability before use:

```
if not YOUR_PACKAGE_AVAILABLE:
 show_status("Error: your-package not installed", is_error=True)
 return
```

### 4. Modify UI Layout

The layout is defined in the `page.add()` section at the bottom of `app.py`. The structure uses Flet containers and rows:

```
page.add(
 ft.Container(
 content=ft.Column([
 # Your UI components here
]),
```

```
 padding=30,
)
)
```

Add your own UI elements:

- `ft.TextField()` - Text input fields
- `ft.Dropdown()` - Dropdown menus
- `ft.Checkbox()` - Checkboxes
- `ft.ElevatedButton()` - Buttons
- `ft.Text()` - Labels and text
- `ft.Row()` and `ft.Column()` - Layout containers

See [Flet documentation](#) for all available controls.

## 5. Persistent Settings

To save additional settings:

```
Save a custom setting
storage.set_ui_state("my_custom_field", "value")

Load a custom setting
value = storage.get_ui_state("my_custom_field", default="default_value")
```

All settings are automatically saved to `~/DART-data/persistent.json`.

## 6. Remove Example Functions

Once you've built your own functions, clean up the examples:

1. Delete function handlers from `app.py`: `on_function_1_list_files`, `on_function_2_count_files`, `on_function_3_system_info`
2. Delete help files: `FUNCTION_1_ANALYZE_ASSETS.md`, `FUNCTION_2_COUNT_FILES.md`, `FUNCTION_3_SYSTEM_INFO.md`
3. Remove entries from `active_functions` list and `functions` dictionary
4. Update the title and description to match your application

## UI Architecture

DART uses OHM's proven layout structure:

- **Collapsible Directories Section:** Saves vertical space once directories are set
- **File Selection:** Always visible for quick file changes between operations
- **Functions Dropdown:** Icon-enhanced with emoji indicators
- **Status Output:** Multi-line with copy-to-clipboard
- **Log Output:** Timestamped entries with copy and clear functionality

- **Help Mode:** Toggle to view function documentation instead of executing (`hdiutil` for macOS, `zip` for all)
- **common-DG-utilities:** Required repository must exist as sibling directory
  - Clone from: <https://github.com/Digital-Grinnell/common-DG-utilities>
  - Expected location: `../common-DG-utilities/` (relative to DART directory)
  - The build scripts automatically include these utilities in the packages
  - If not found, the build will show a warning but continue

### What the installers do:

- **macOS DMG:** Creates an auto-installer that installs to `~/DART/` (no admin required)
- **Windows ZIP:** Packages with auto-installer:

```
bash build_dmg.sh 2.0
```

This creates `DART_v2.0.dmg` with:

- Auto-installer script (`Install DART.command`)
- Installs to `~/DART/` (no admin permissions required)
- Includes common-DG-utilities automatically
- Creates desktop launcher
- No code signing (users must right-click → Open on first use)

### Distribution:

- Send the DMG file to users
- Users double-click the installer
- Installation completes automatically
- No manual file copying required Create a distributable DMG file: `DART_v2.0_Windows.zip`` with:
- All source files including common-DG-utilities
- `scripts/run.bat` launcher
- Automatic dependency installation on first launch
- No admin permissions required

### Distribution:

- Send the ZIP file to users
- Users extract and run `scripts/run.bat`
- First launch installs dependencies automatically

Recipients need Python 3.8+ installed with "Add Python to PATH" checked

- Self-contained app bundle
- Automatic dependency installation on first launch
- No code signing (users must right-click → Open on first launch)

### Windows ZIP

Create a distributable ZIP package:

```
bash build_windows_zip.sh 1.0
```

This creates `YourApp_v1.0_Windows.zip` with:

- All source files
- `scripts/run.bat` launcher
- Automatic dependency installation on first launch

Recipients need Python 3 installed (one-time setup).

## Logging

All application activity is logged to:

```
{working_folder}/logfiles/dart_YYYYMMDD_HHMMSS.log
```

Log files are stored in your working/outputs folder (set in the UI) to keep logs organized with the project data they relate to. On initial startup before a working folder is set, logs temporarily go to `~/DART-data/logfiles/`.

Use the logger in your functions:

```
logger.info("Information message")
logger.warning("Warning message")
logger.error("Error message")
logger.debug("Debug message")
```

Console output shows only errors; all levels are written to log files.

## Help Documentation

Help files use GitHub Flavored Markdown and support:

- Headers (`#`, `##`, `###`)
- Lists (ordered and unordered)
- Code blocks with syntax highlighting
- Tables
- Links
- **Bold** and *italic* text

Create help documentation for each function to guide users.

## Examples of Apps Built with This Template

- **OHM - Oral History Manager**: Audio processing workflow for digital archives

- (Add your own app here!)

## Tips for Development

### Testing Your Changes

After modifying `app.py`, just rerun:

```
./run.sh # or scripts\run.bat on Windows
```

The virtual environment and dependencies are cached, so subsequent runs are fast.

### Debugging

- Check log files in `{working_folder}/logfiles/` for errors (or `~/DART-data/logfiles/` before working folder is set)
- Console shows error-level messages immediately
- Use `logger.debug()` for detailed troubleshooting

### Version Control

Initialize a git repository for your new app:

```
git init
git add .
git commit -m "Initial commit based on DART"
```

The included `.gitignore` excludes:

- Virtual environments (`.venv/`)
- Python cache (`__pycache__`)
- Log files
- Build artifacts

## Metadata Field Renaming Tools

DART includes utility scripts for coordinating metadata field name changes across CSV files and CollectionBuilder configuration.

To stay in sync with Digital-Grinnell/collectionbuilder-csv and its upstream CollectionBuilder CSV repository, DART now treats plain field names such as `title`, `description`, and `date` as canonical. If an older project still has `dc_`-prefixed CSV headers or config fields, remove those prefixes before continuing.

`scripts/rename_metadata_field.py`

Python script for renaming individual fields with automatic backup and preview:

```
Preview changes (dry run)
python3 scripts/rename_metadata_field.py \
 --csv metadata.csv \
 --old-field dc_title \
 --new-field title

Apply changes to CSV and CollectionBuilder
python3 scripts/rename_metadata_field.py \
 --csv metadata.csv \
 --old-field dc_title \
 --new-field title \
 --cb-dir ../collectionbuilder \
 --apply
```

### Features:

- Safe dry-run preview mode (default)
- Field name validation (prevents colons, periods, and other problematic characters)
- Rejects legacy `dc_` target names so exports stay aligned with CollectionBuilder CSV
- Automatic timestamped hidden backups (dotted filenames)
- Updates CSV headers and CollectionBuilder config files
- Pattern matching for Liquid metadata variables (`item.field`), YAML in config files
- **Does NOT change `page.title`** (page front matter) - only changes `item.title` (metadata)
- **Supports CSV-based configs:** Updates config-browse.csv, config-metadata.csv, etc.
- Clear reporting of what changed

**Fix script** for CSV-based CollectionBuilder configs (if main script missed them):

```
Update CSV config files after removing legacy dc_ prefixes
python3 scripts/fix_config_csv_fields.sh ~/GitHub/collectionbuilder
```

This fixes the 'field' column in config-browse.csv, config-metadata.csv, etc.

`scripts/batch_rename_dublin_core.sh`

Bash script for batch normalizing legacy `dc_` field names back to standard CollectionBuilder CSV fields:

```
Remove legacy dc_ prefixes from all standard fields
bash scripts/batch_rename_dublin_core.sh metadata.csv ../collectionbuilder
```

### Renames these fields (if present):

- `dc_title` → `title`
- `dc_description` → `description`
- `dc_creator` → `creator`
- `dc_subject` → `subject`

- `dc_date` → `date`
- `dc_format` → `format`
- `dc_rights` → `rights`
- Plus more (see `RENAME_METADATA_FIELD.md`)

### Features:

- Previews all changes before applying
- Prompts for confirmation
- Processes multiple fields in one operation
- Creates backups for all modified files

### `scripts/migrate_legacy_working_files.py`

One-time helper for older projects that still have DART-generated artifacts outside `.DART-working-directory`.

```
Preview what would be moved (dry run)
python3 scripts/migrate_legacy_working_files.py /path/to/working-folder

Apply migration
python3 scripts/migrate_legacy_working_files.py /path/to/working-folder --
apply
```

This migrates legacy files such as `dart_settings.json`, `DART_*`, `csvdiff_*`, and `*.backup_*` into `.DART-working-directory/legacy-migrated/`. It only targets DART/csvdiff HTML report-style names and does not move arbitrary project HTML files.

 **Full documentation:** See [RENAME\\_METADATA\\_FIELD.md](#)

## Flet Resources

- **Documentation:** <https://flet.dev/docs/>
- **Controls Gallery:** <https://flet.dev/docs/controls>
- **GitHub:** <https://github.com/flet-dev/flet>
- **Discord:** <https://discord.gg/dzWXP8SHG8>

## License

MIT License - See [LICENSE](#) or [LICENSE.md](#) for full details.

Copyright (c) 2026 Digital.Grinnell / DART Contributors

Free to use, modify, and distribute. Attribution appreciated but not required.

## Contributing

Contributions are welcome! Please feel free to:

- Report bugs or issues

- Suggest new features or improvements
- Submit pull requests
- Share applications you've built with DART

## About

DART (Digital Asset Routing and Transformation) was created by extracting and generalizing the proven UI framework from OHM (Oral History Manager). It provides a professional platform for Flet desktop applications, with robust support for settings persistence, logging, function management, and help systems.

The application architecture has been refined through real-world production use, ensuring reliability and maintainability.

**Repository:** <https://github.com/Digital-Grinnell/DART>

Built with ❤️ using [Flet](#) by the Digital.Grinnell team.